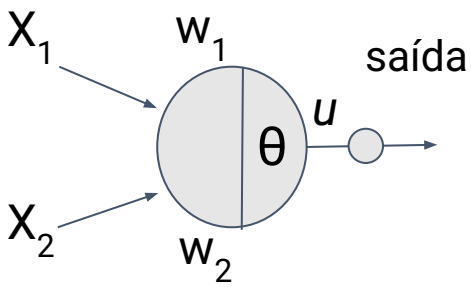
Sistemas Inteligentes

Profa. Dra. Nathalia da Cruz Alves

Perceptron

Limitações de um único neurônio

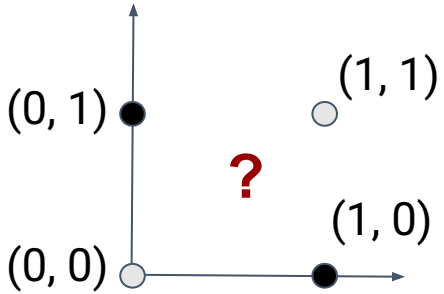
entradas



$$u = x_1.w_1 + x_2.w_2 + x_0.\theta$$

Determine w_1, w_2 e θ para que o Perceptron aprenda a função lógica **OU-EXCLUSIVO (XOR)**

saída = $f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$



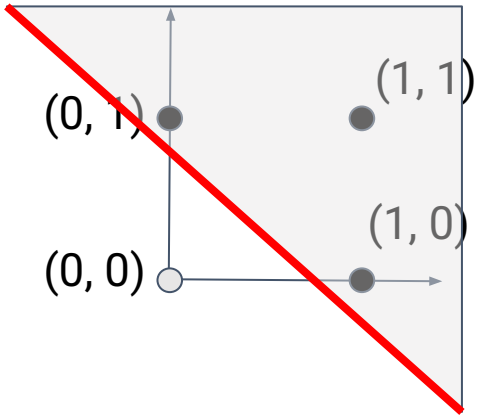
O Perceptron é capaz de distinguir **apenas padrões linearmente separáveis**

Ver demonstração Perceptron com XOR

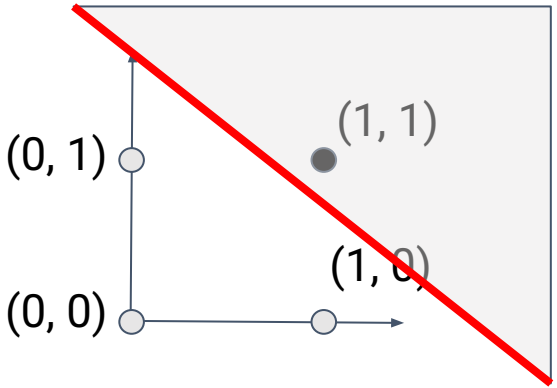
Perceptron

Especialização de neurônios

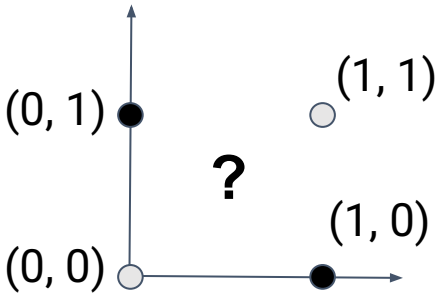
OR



AND



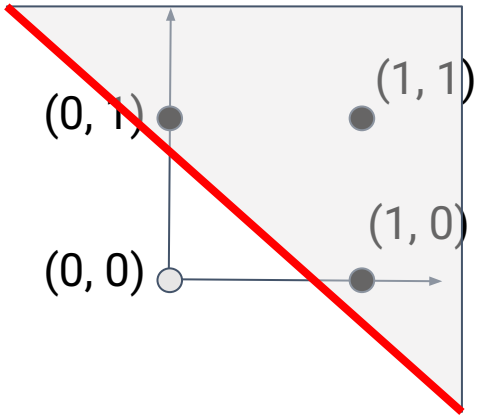
XOR



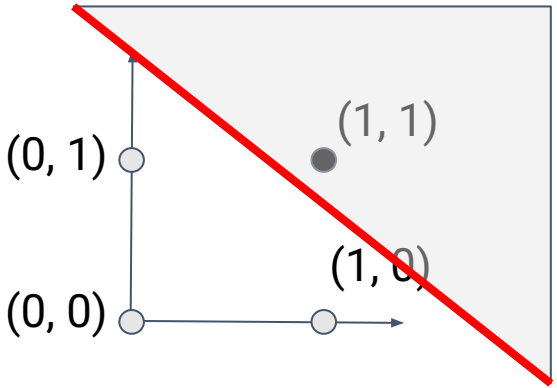
Perceptron

Especialização de neurônios

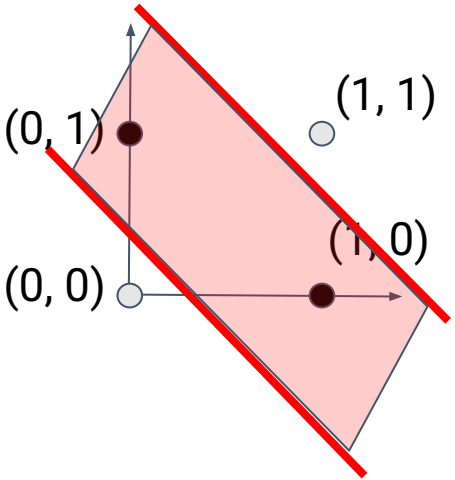
OR



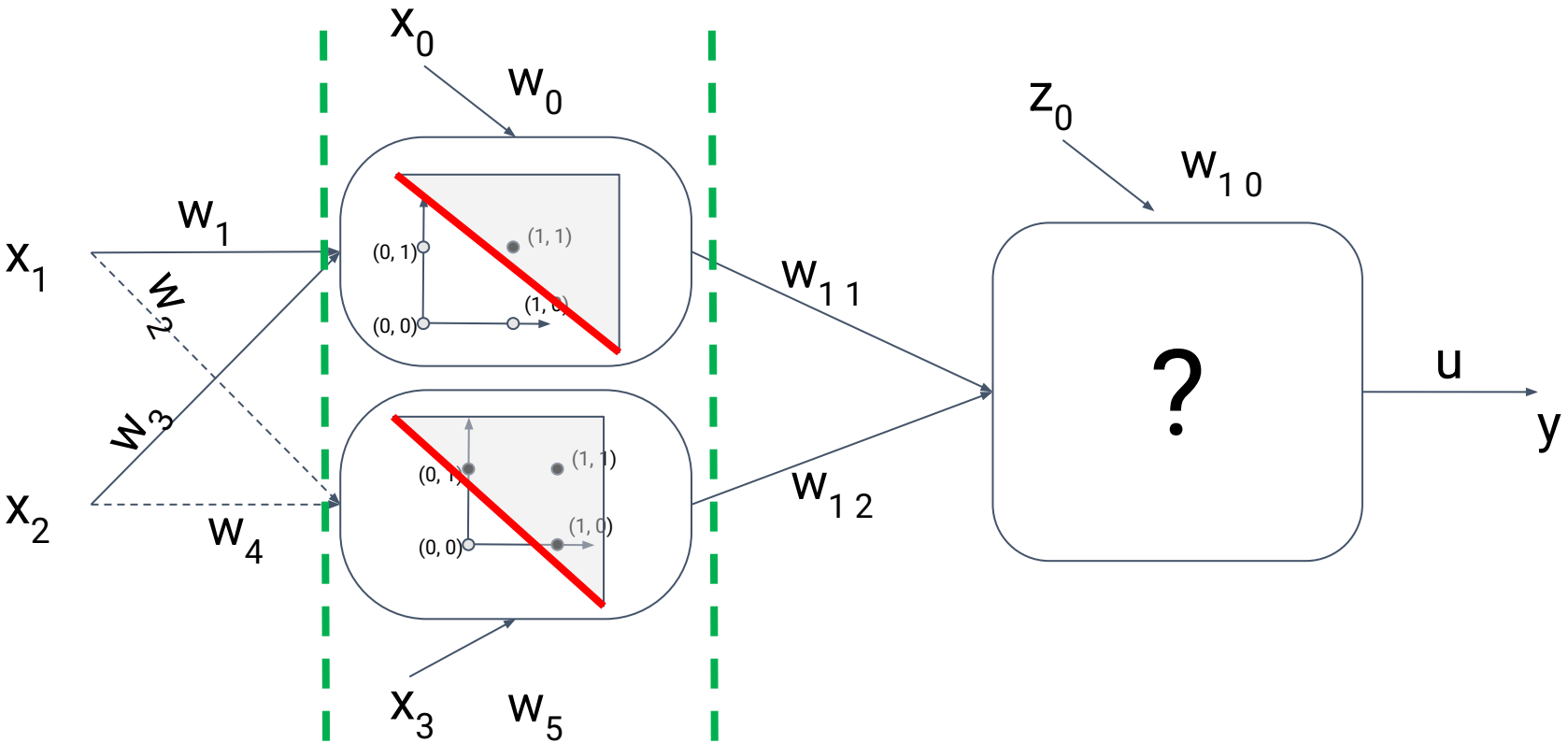
AND



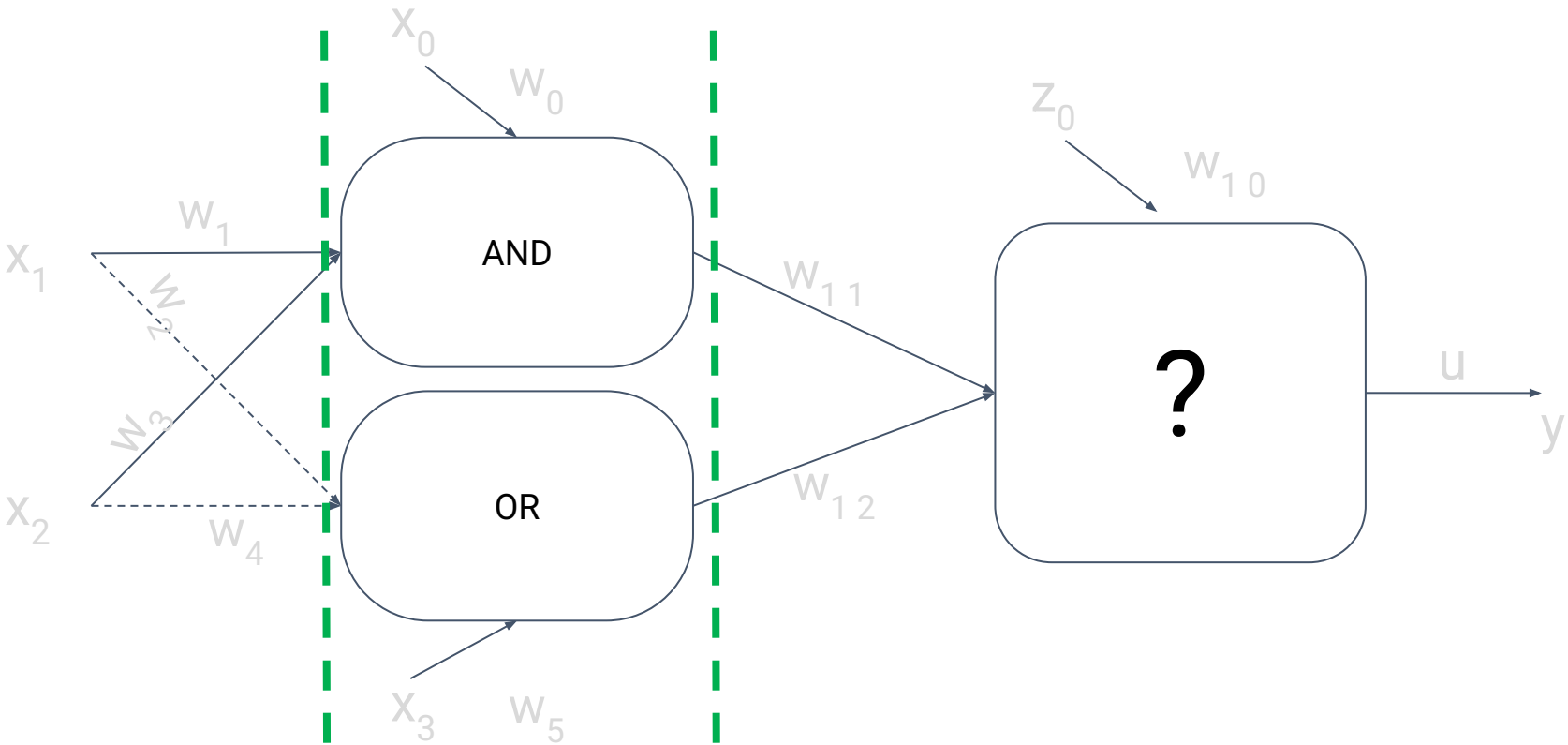
XOR



MLP (*MultiLayer Perceptrons*)



MLP (*MultiLayer Perceptrons*)



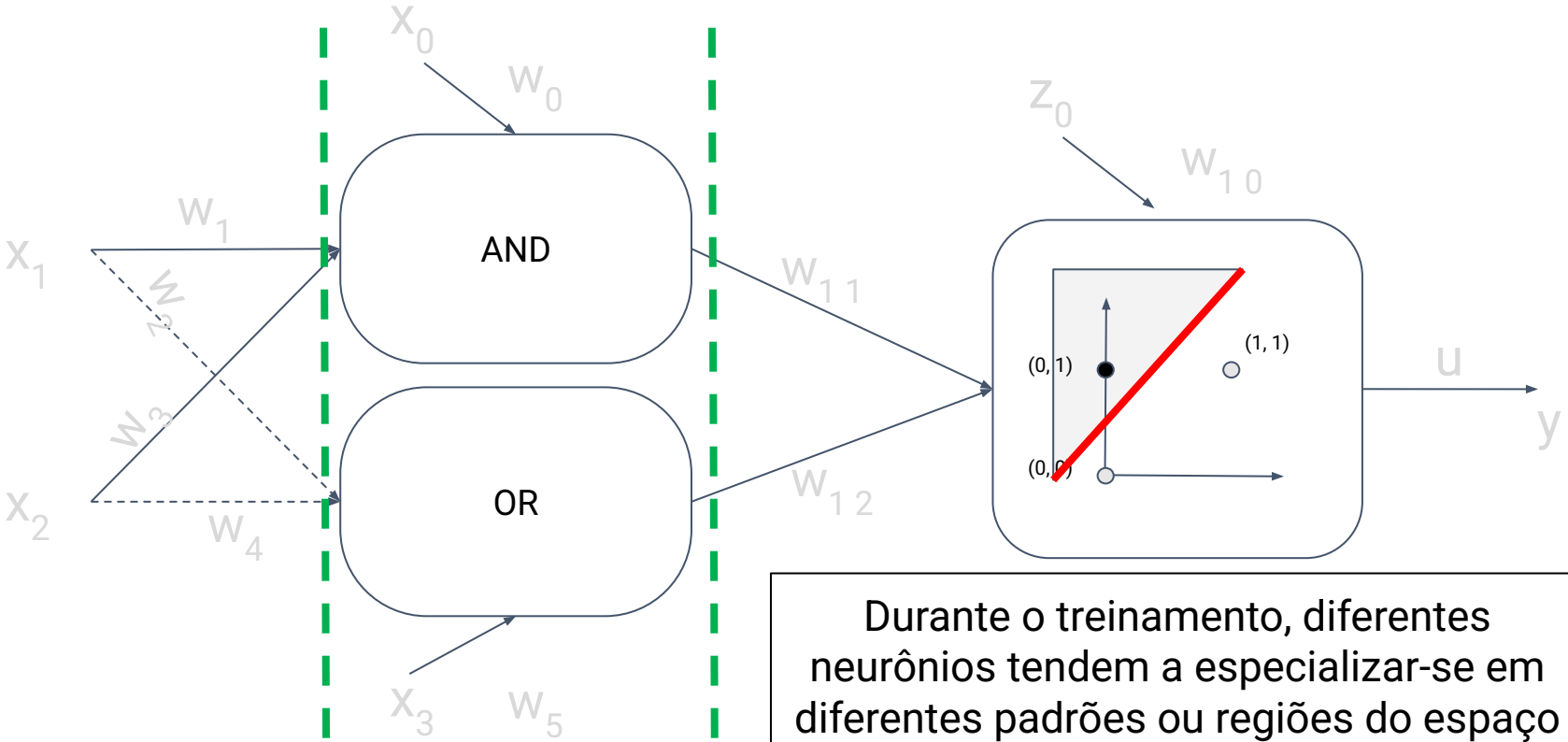
MLP (*MultiLayer Perceptrons*)

Redução de complexidade com especialização de neurônios

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

MLP (*MultiLayer Perceptrons*)

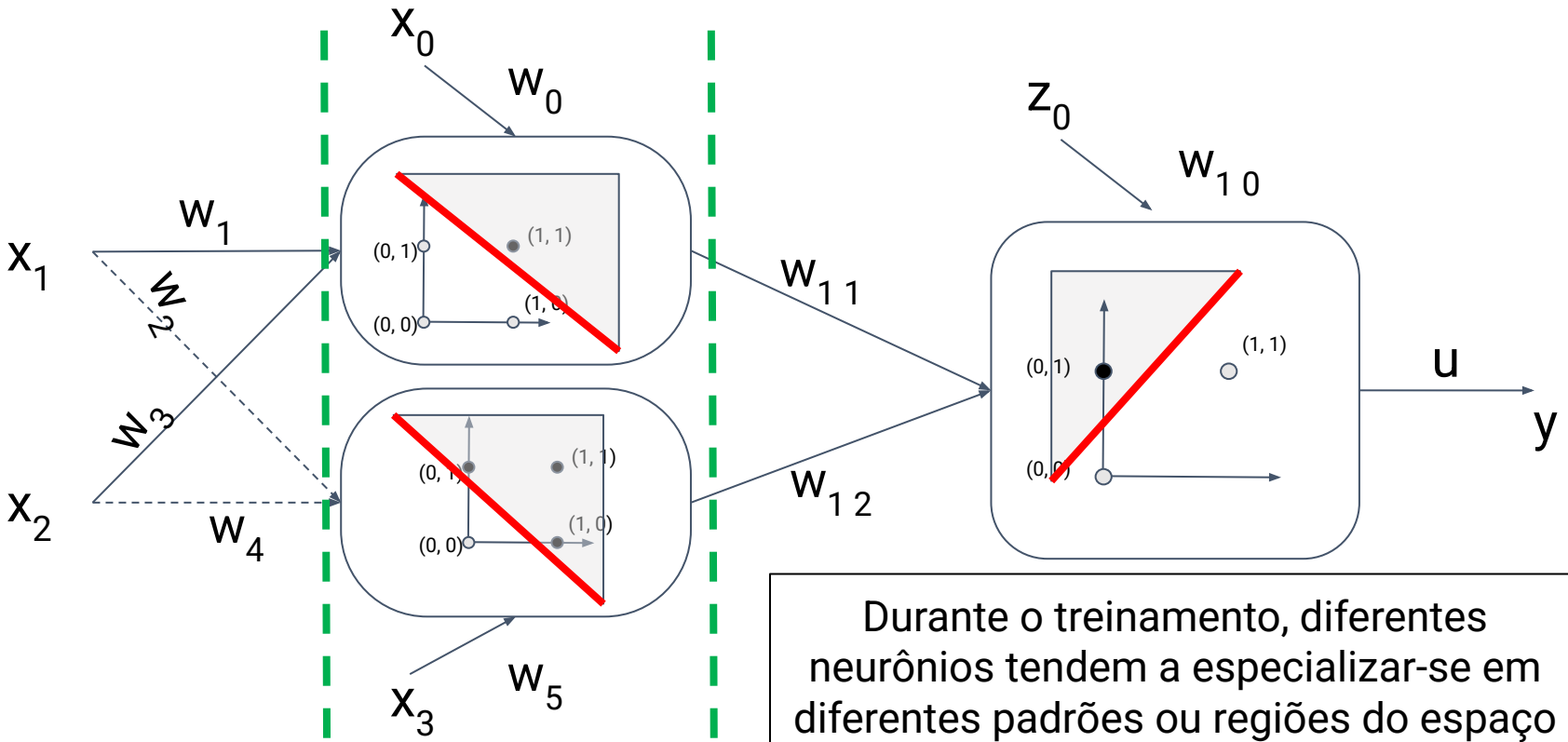
Redução de complexidade com especialização de neurônios



Durante o treinamento, diferentes neurônios tendem a especializar-se em diferentes padrões ou regiões do espaço

MLP (*MultiLayer Perceptrons*)

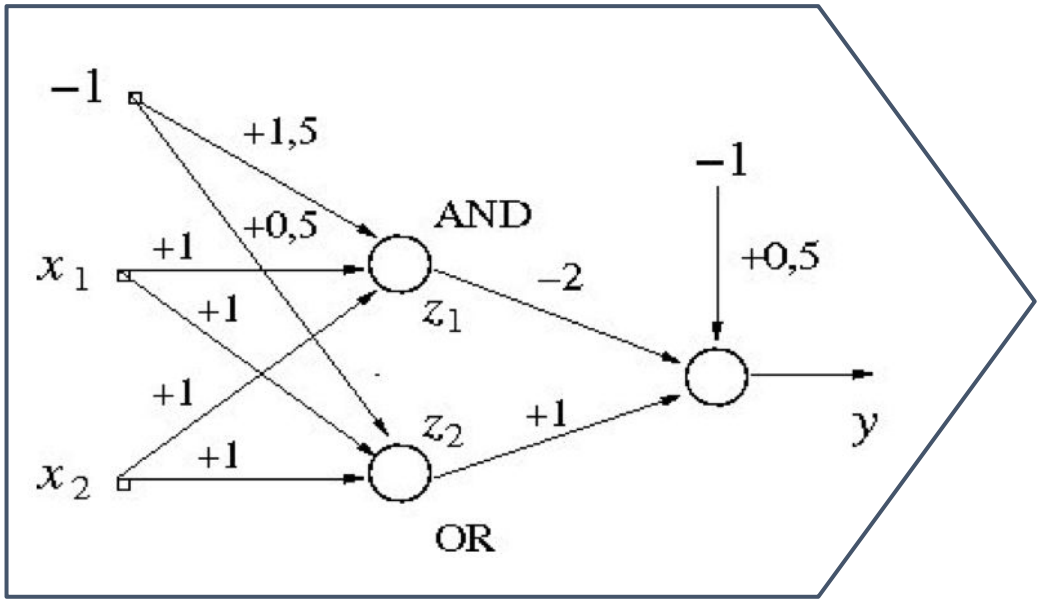
Redução de complexidade com especialização de neurônios



Durante o treinamento, diferentes neurônios tendem a especializar-se em diferentes padrões ou regiões do espaço

MLP (*MultiLayer Perceptrons*)

Redução de complexidade com especialização de neurônios



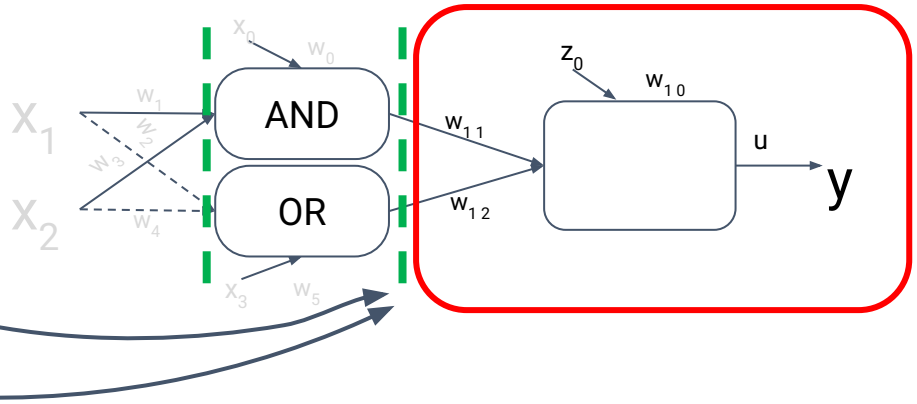
Uma possível solução

Ver demonstração Perceptron com camada oculta para o XOR

Como atualizar os pesos dos neurônios das camadas ocultas?

Não é possível **calcular o erro** (*d* - *y*) do quanto os **pesos dos neurônios das camada de entrada** (w_i) estão contribuindo para o erro, pois há uma camada oculta com neurônios intermediários

$$w_i(t+1) = w_i(t) + \eta \cdot (d - y) \cdot x_i$$



É necessária alguma **estimativa** do erro

Como atualizar os pesos dos neurônios das camadas ocultas?

A contribuição das camadas ocultas para o erro **não é observável diretamente**. Por isso, precisa ser inferida matematicamente:

Backpropagation

- Calcula derivadas parciais do erro em relação a cada peso, permitindo ajustar os parâmetros na direção de menor erro
- Retropropaga os erros de cada neurônio

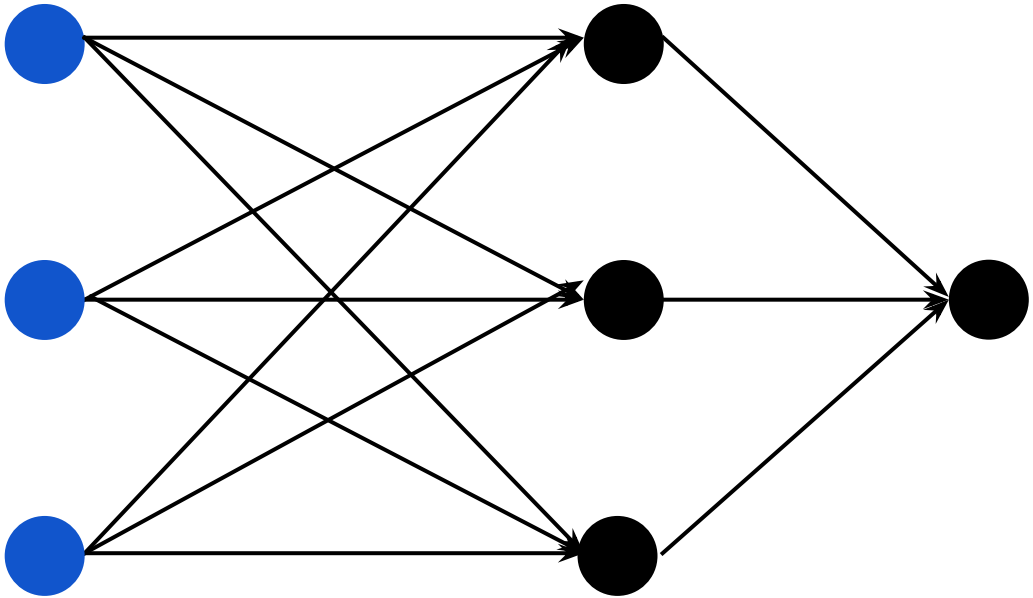
Descida em Gradiente (*Gradient Descent*)

- Calcula o gradiente (conjunto das derivadas parciais) do erro em relação aos pesos da rede

Backpropagation

Algoritmo usado para calcular o erro de cada neurônio da rede.

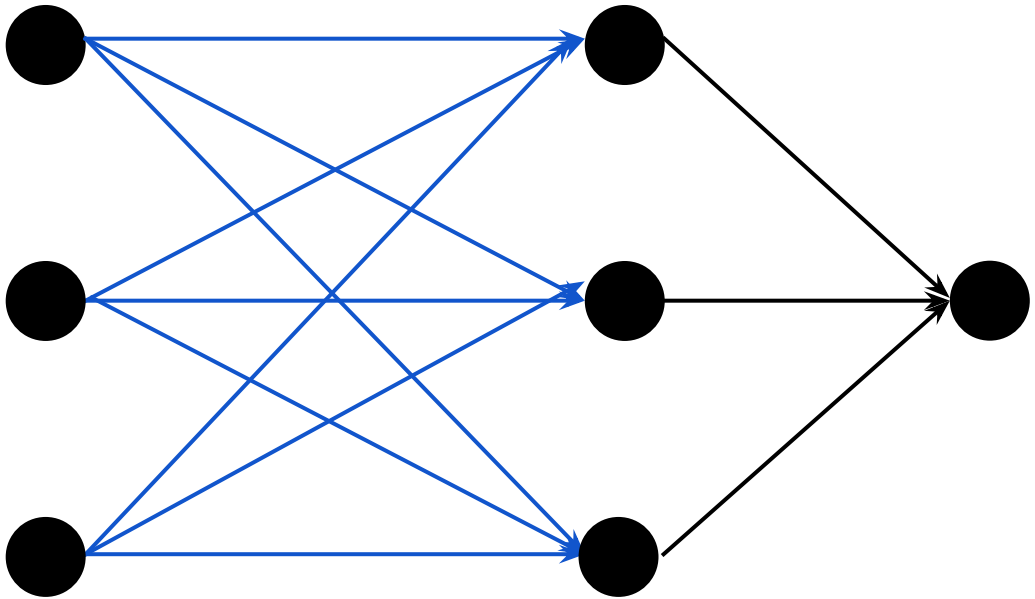
A informação entra na rede, passa pelas camadas multiplicada pelos pesos, e gera uma previsão



Backpropagation

Algoritmo usado para calcular o erro de cada neurônio da rede.

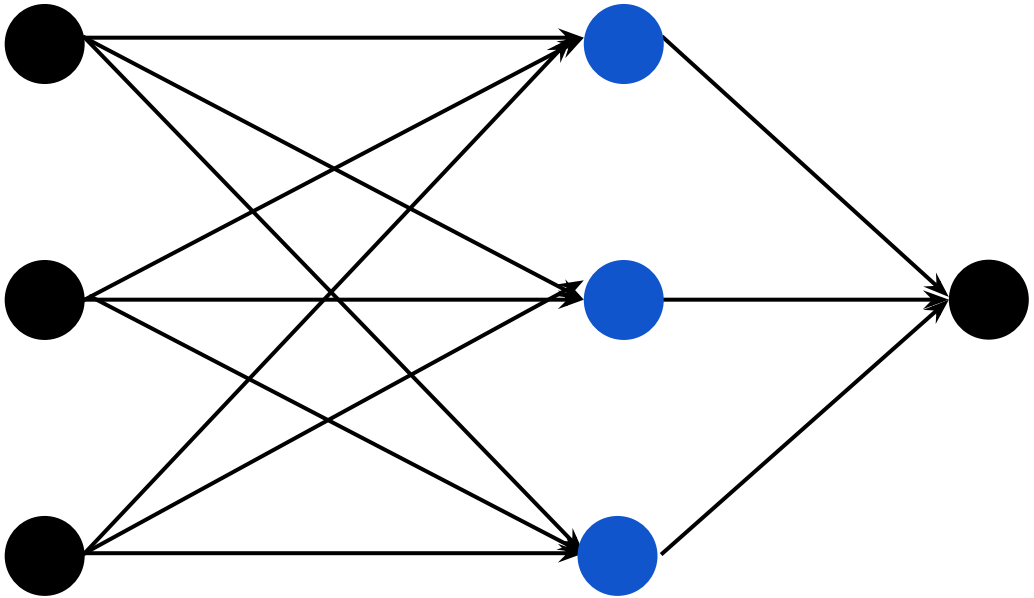
A informação entra na rede, passa pelas camadas multiplicada pelos pesos, e gera uma previsão



Backpropagation

Algoritmo usado para calcular o erro de cada neurônio da rede.

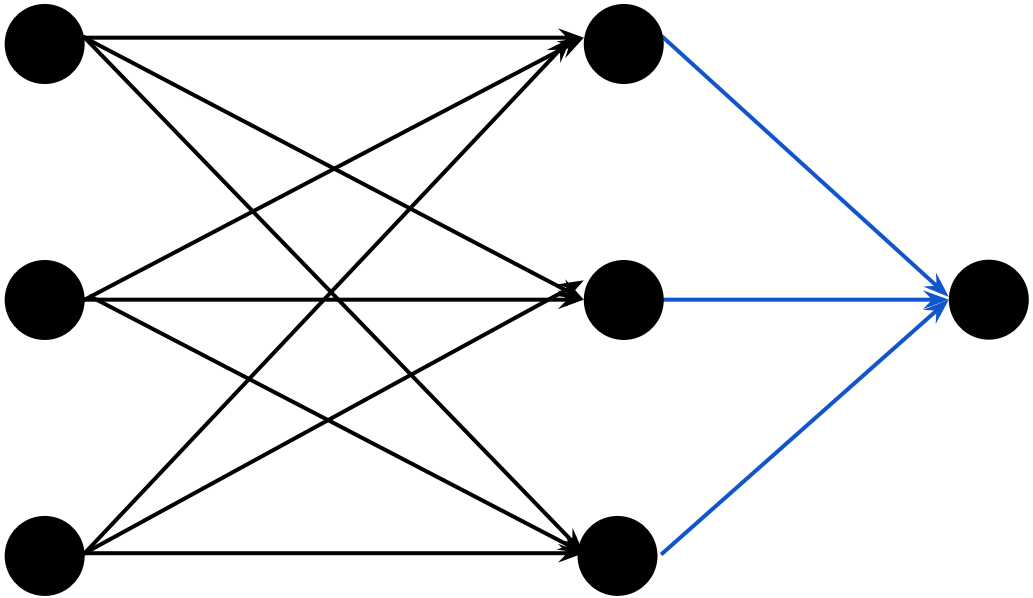
A informação entra na rede, passa pelas camadas multiplicada pelos pesos, e gera uma previsão



Backpropagation

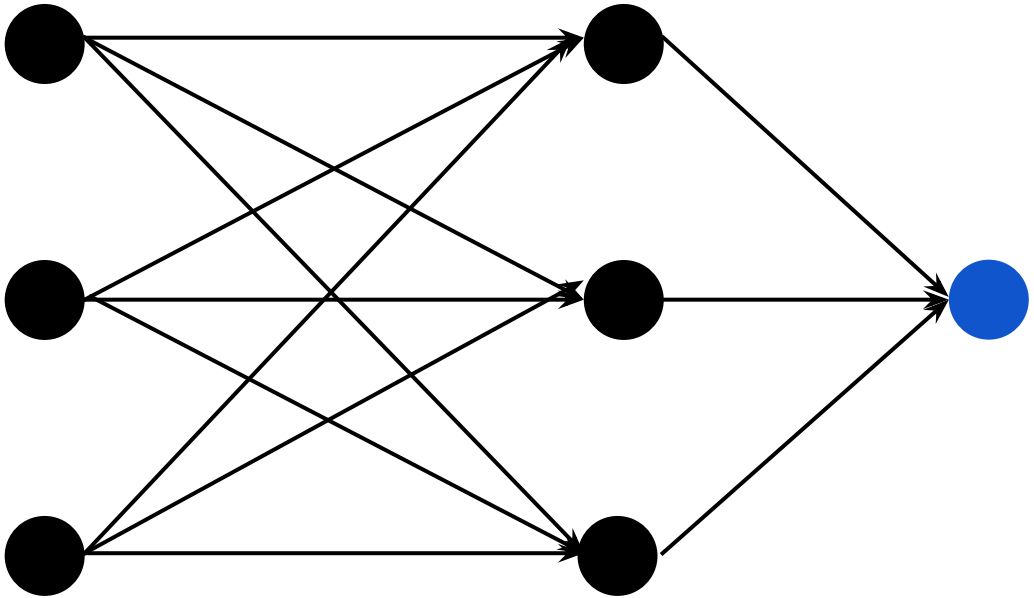
Algoritmo usado para calcular o erro de cada neurônio da rede.

A informação entra na rede, passa pelas camadas multiplicada pelos pesos, e gera uma previsão



Backpropagation

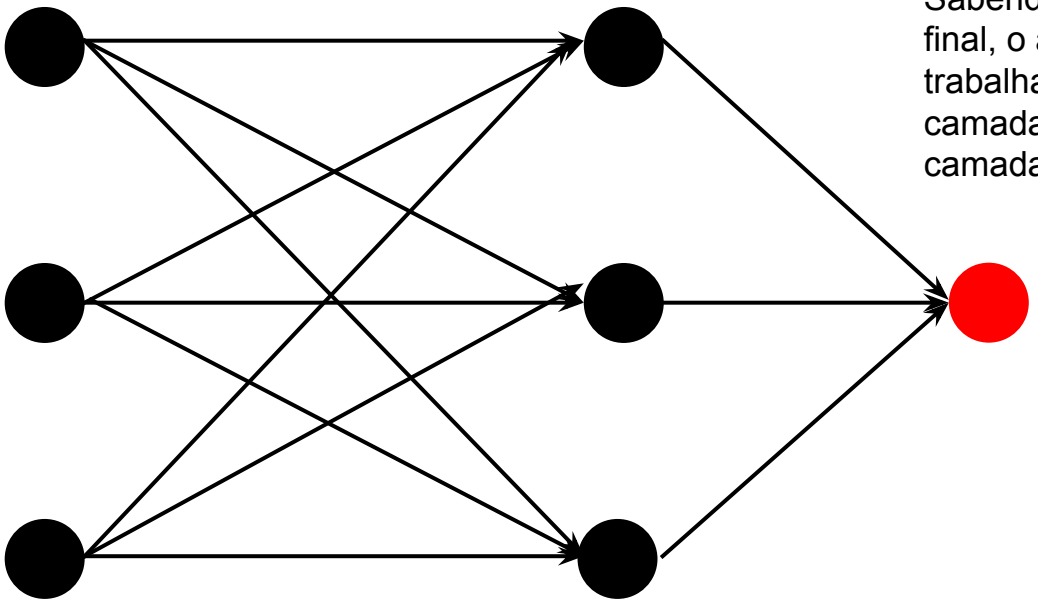
Algoritmo usado para calcular o erro de cada neurônio da rede.



A função de erro compara essa previsão com o valor real e diz o tamanho do erro (ex: "a rede errou por 0.5")

Backpropagation

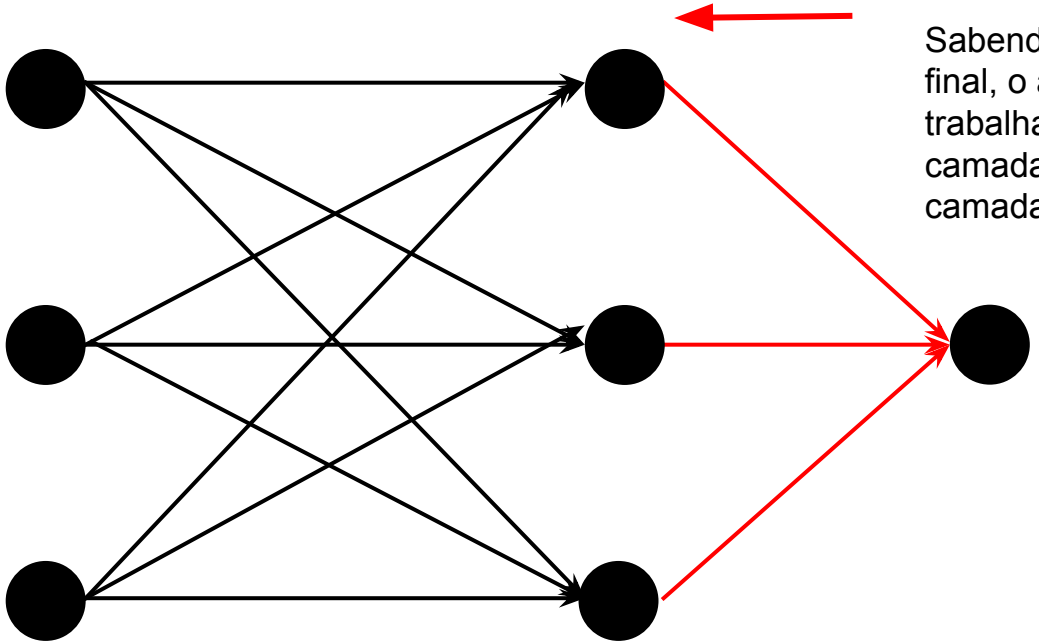
Algoritmo usado para calcular o erro de cada neurônio da rede.



Sabendo o tamanho do erro no final, o algoritmo começa a trabalhar de trás para frente (da camada de saída em direção à camada de entrada)

Backpropagation

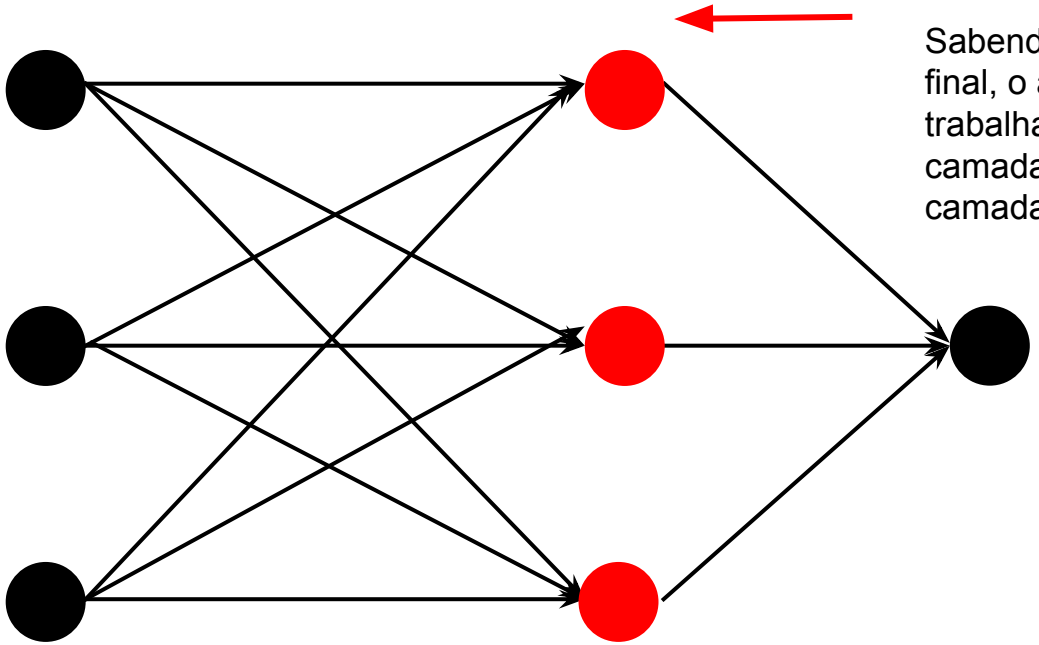
Algoritmo usado para calcular o erro de cada neurônio da rede.



Sabendo o tamanho do erro no final, o algoritmo começa a trabalhar de trás para frente (da camada de saída em direção à camada de entrada)

Backpropagation

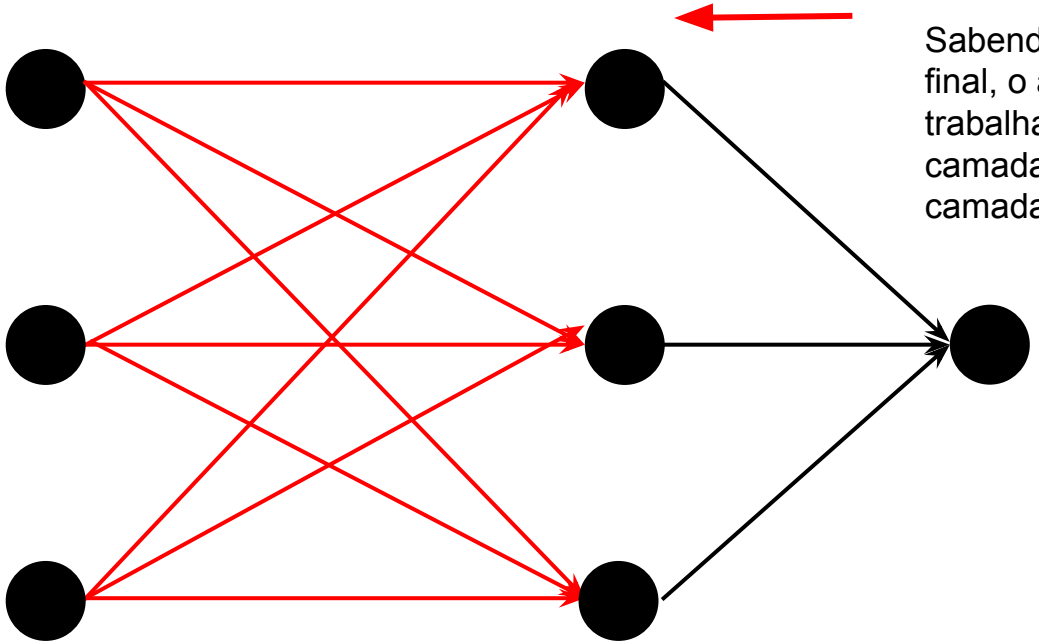
Algoritmo usado para calcular o erro de cada neurônio da rede.



Sabendo o tamanho do erro no final, o algoritmo começa a trabalhar de trás para frente (da camada de saída em direção à camada de entrada)

Backpropagation

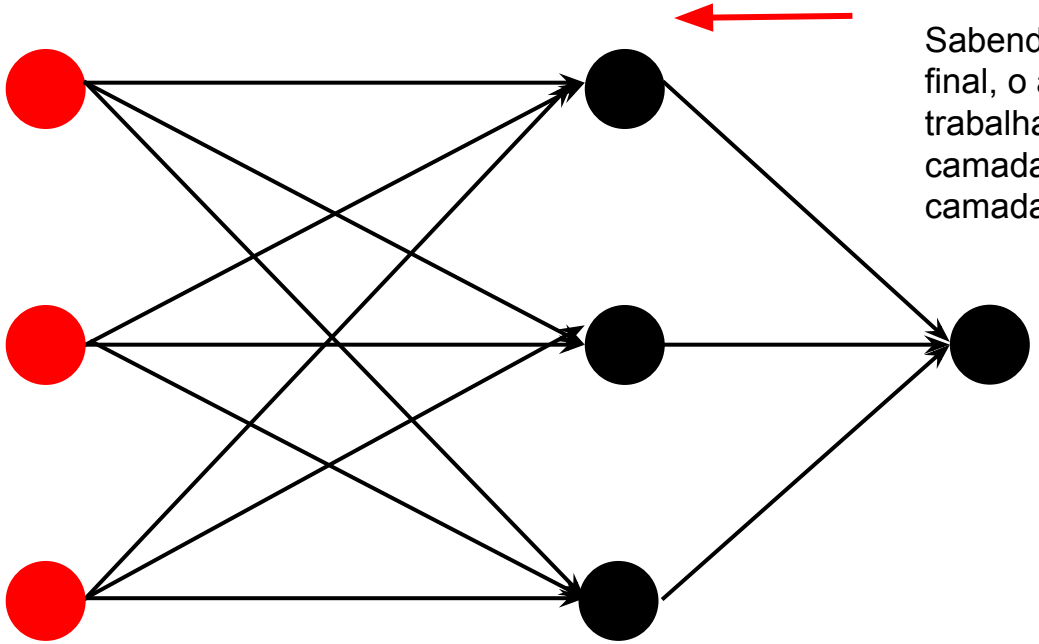
Algoritmo usado para calcular o erro de cada neurônio da rede.



Sabendo o tamanho do erro no final, o algoritmo começa a trabalhar de trás para frente (da camada de saída em direção à camada de entrada)

Backpropagation

Algoritmo usado para calcular o erro de cada neurônio da rede.



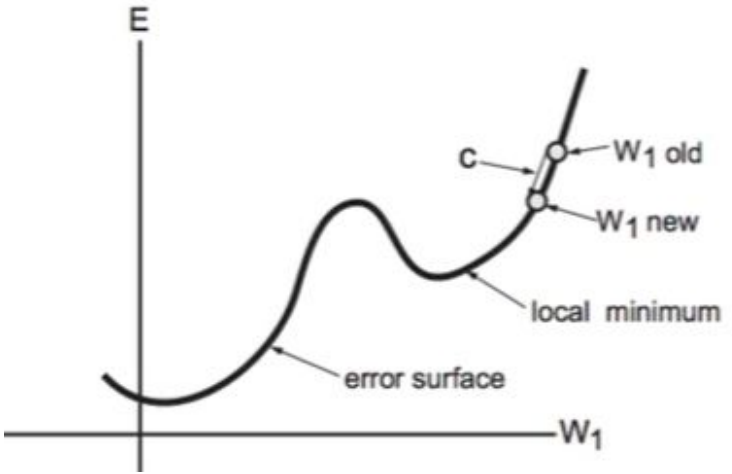
Sabendo o tamanho do erro no final, o algoritmo começa a trabalhar de trás para frente (da camada de saída em direção à camada de entrada)

Treinamento de uma MLP

- Cada conexão entre os neurônios da rede possui um **peso** (um valor numérico que define a força dessa conexão)

- A **função de erro** mede a **distância** entre a resposta que a rede neural deu e a resposta correta que ela deveria dar

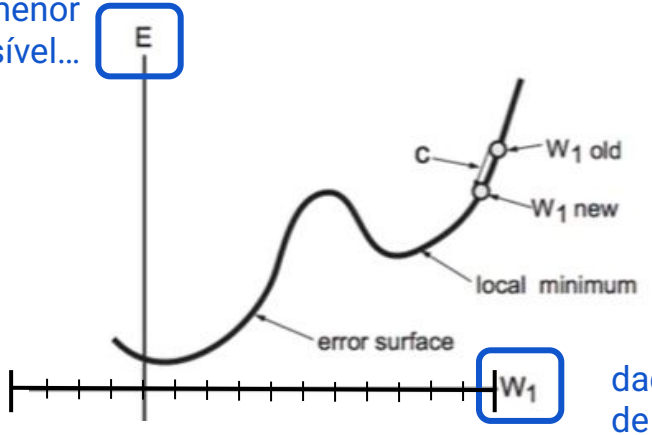
- Cada **combinação** diferente de pesos gera um valor diferente para a função de erro
  - Se os pesos estão ruins, o erro é alto
  - Se estão bons, o erro é baixo



Treinamento de uma MLP

Com a **derivada**, é possível saber exatamente se o erro está aumentando ou diminuindo com base nos pesos atuais, além de indicar quão rápido essa mudança está acontecendo

Qual é o menor erro possível...



dado um conjunto de pesos?

Descida em Gradiente (*Gradient Descent*)

Algoritmo usado para atualizar os pesos da rede neural.

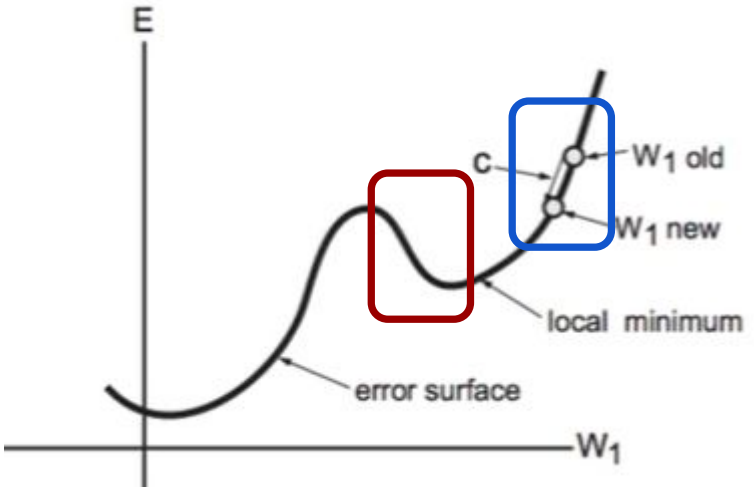
- O gradiente é o conjunto das derivadas parciais do erro em relação aos pesos da rede
  - Mede a inclinação do erro e a direção em que ele aumenta mais rapidamente
- O objetivo é reduzir progressivamente o erro

Se a **derivada for positiva**

- O erro aumenta quando o peso aumenta
  - Então o peso deve **diminuir**

Se a **derivada for negativa**

- O erro diminui quando o peso aumenta
  - Então o peso deve **aumentar**



Funções de Ativação Não Lineares

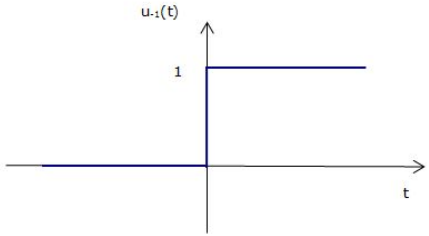
Se todas as camadas de uma MLP utilizarem apenas operações lineares, a rede continua equivalente a um único Perceptron linear.

As funções de ativação **não lineares** (sigmoide, tanh, ReLU, Leaky ReLu, PReLU, ELU, SELU, etc.):

- Permitem representar padrões complexos
- Tornam possível resolver problemas como o XOR

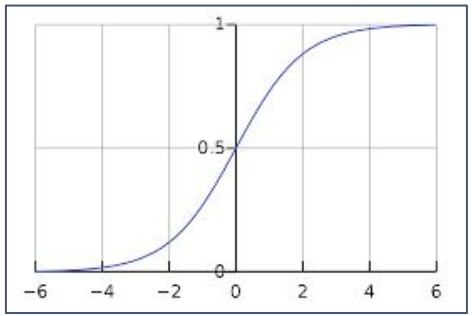
Funções de Ativação Não Lineares

$$f(u) = \begin{cases} 1, \text{ se } u > 0 \\ 0, \text{ se } u \leq 0 \end{cases}$$



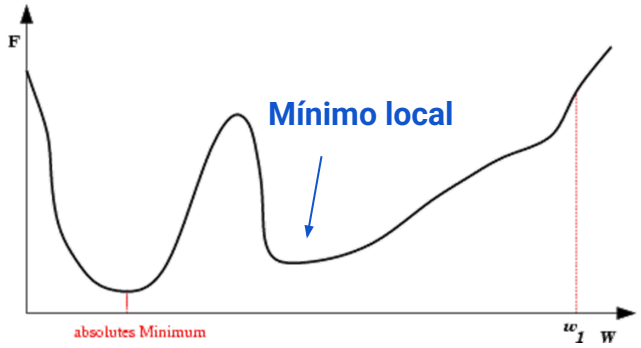
A função degrau tem a **derivada nula** na maior parte do domínio. Isso impede o uso direto de gradiente.

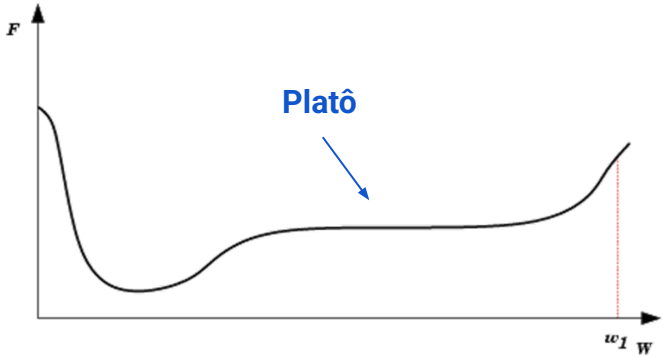
$$f(x) = \frac{1}{1+e^{-x}}$$

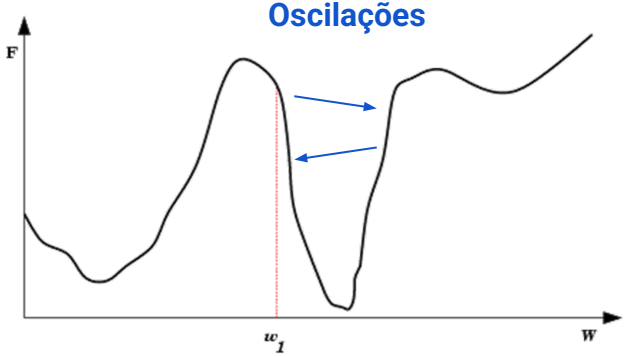


A função logística é suave e contínua, com uma derivada diferente de zero na maior parte do seu caminho.

Descida em Gradiente (*Gradient Descent*)

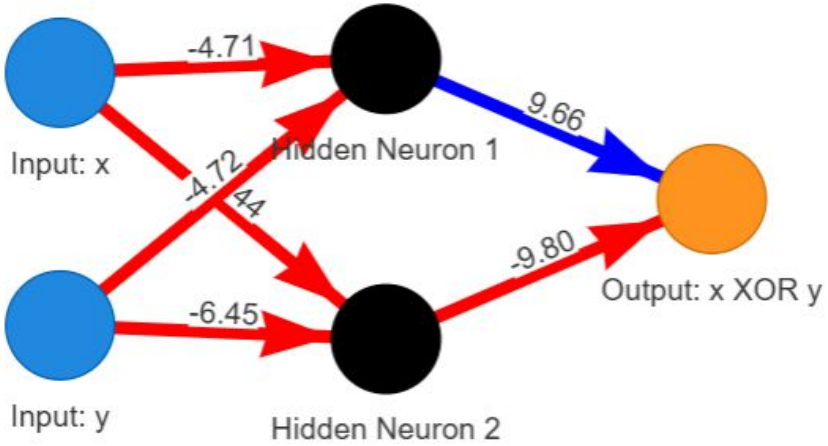




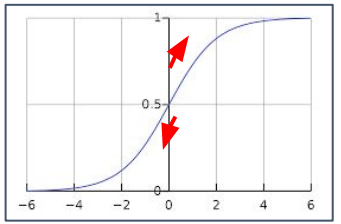




Demonstração MLP
XOR



$$f(x) = \frac{1}{1+e^{-x}}$$



Inputs		Expected Output	Actual Output
x	y	x XOR y	x XOR y
0	0	0	0.017
0	1	1	0.982
1	0	1	0.982
1	1	0	0.023

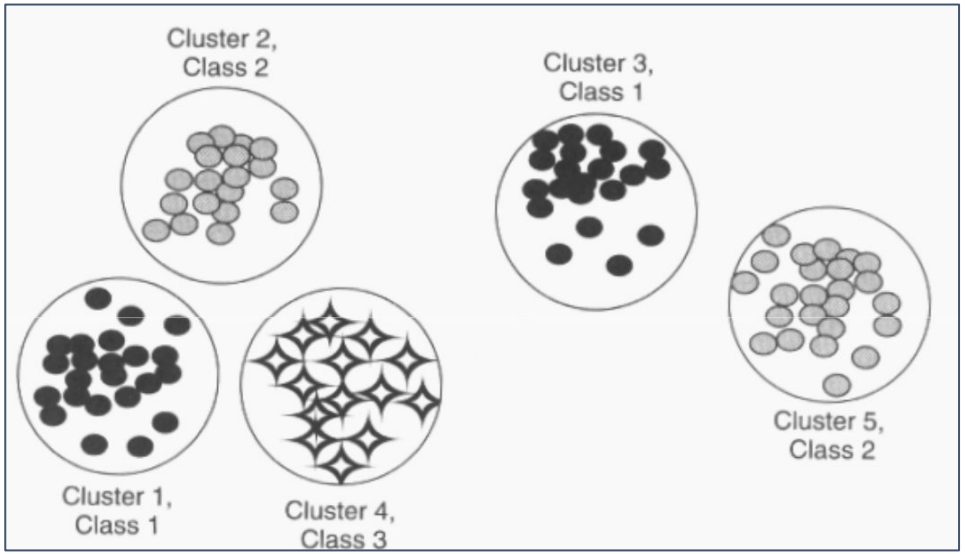
Veja demonstração MLP **com camada oculta** para o XOR (com sigmoidal vs. degrau)

Demonstração MLP

Problemas não linearmente separáveis

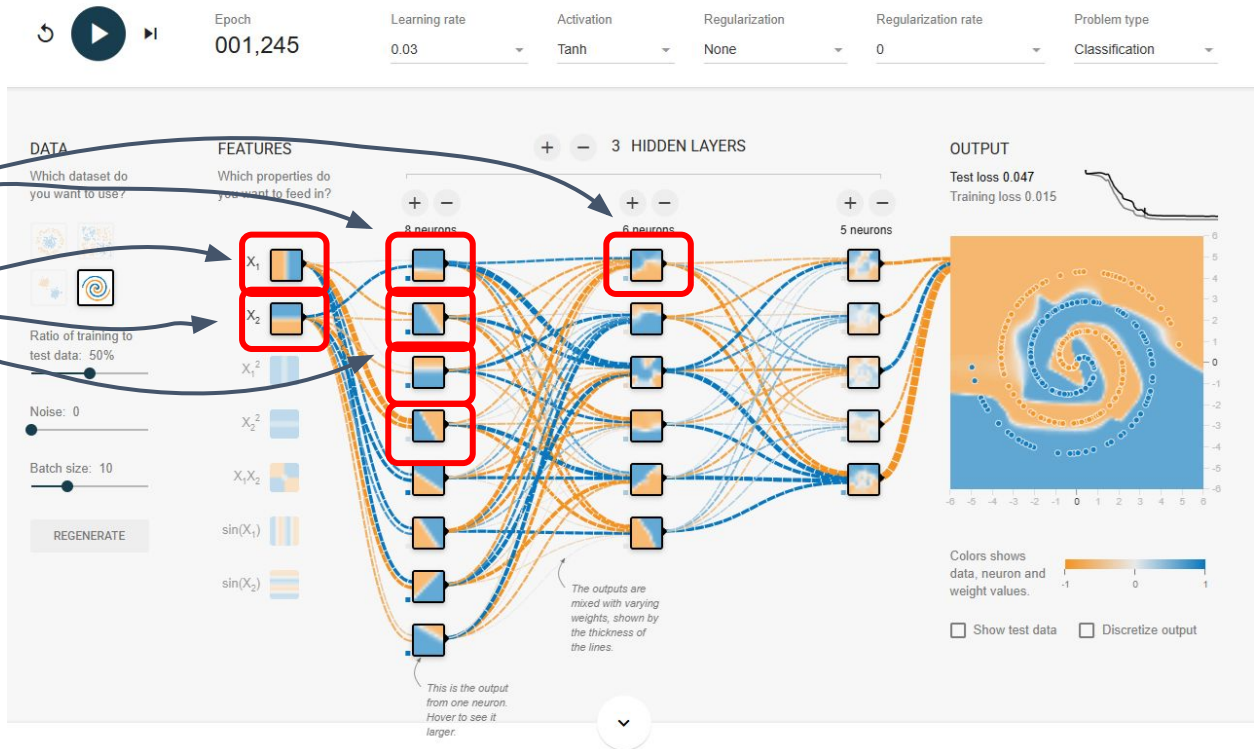
Em problemas bidimensionais, cada neurônio pode ser interpretado como **uma reta separando regiões do espaço**

Veja demonstração Perceptron com dados não lineares



Demonstração MLP

Neurônios especializados em diferentes padrões

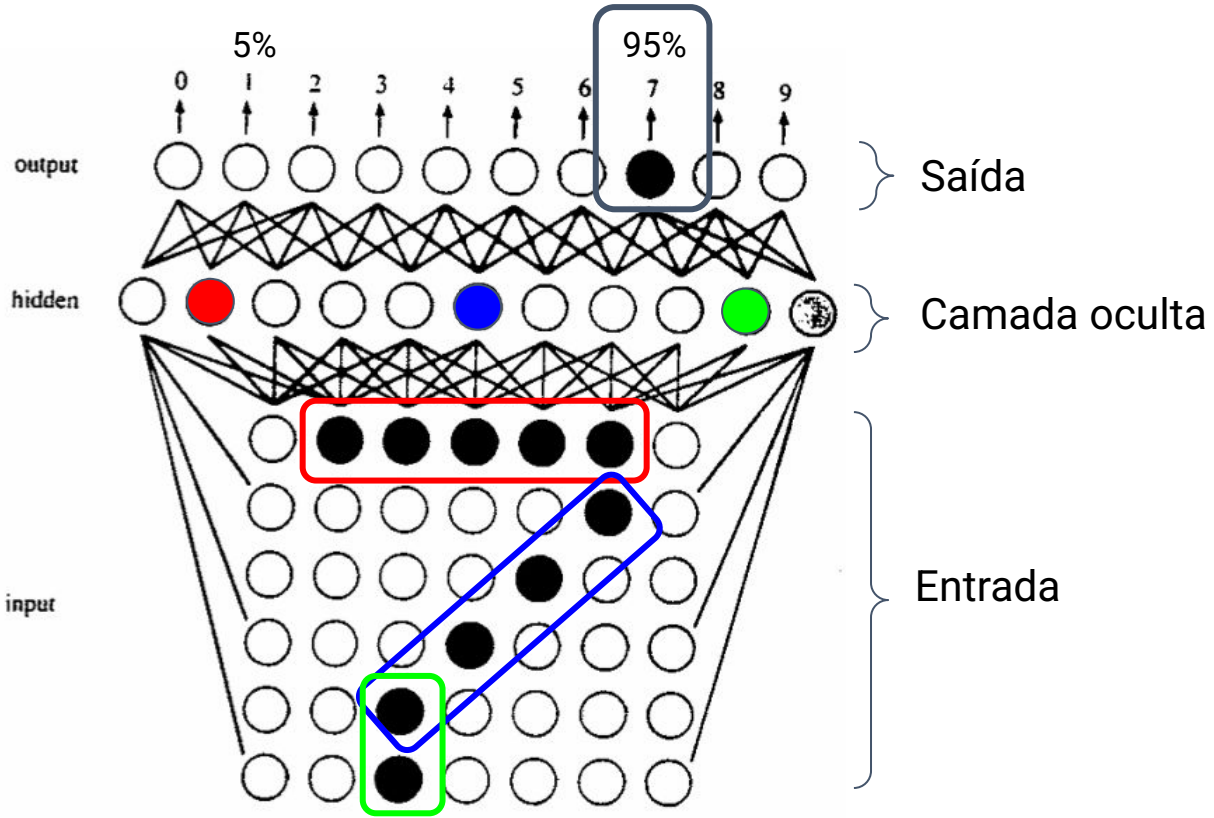


Ver TensorFlow Playground

O Papel das Camadas Ocultas

Exemplo: detector de números

- Durante o treinamento, diferentes neurônios tendem a especializar-se em diferentes padrões ou regiões do espaço
- Dependendo do quão “ativado” estiver um neurônio, é possível dar uma resposta



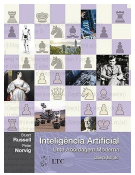
Camadas Ocultas

Quantas camadas utilizar? Quantos neurônios utilizar?

- Testes
- Experimentos
- Literatura

Veja exemplo com diferentes classes

Leia mais sobre este tema...



Russell, S., Norvig, Peter. Inteligência Artificial - Capítulo 18. 2ª Ed. / Capítulo 19. 4ª Ed. Campus-Elsevier; Brasil, 2022.

creative commons

COMMONS DEED

Atribuição-Uso Não-Comercial-Compartilhamento pela Licença 2.5 Brasil

Você pode:

- copiar, distribuir, exibir e executar a obra

- criar obras derivadas

Sob as seguintes condições:

Atribuição — Você deve dar crédito ao autor original, da forma especificada pelo autor ou licenciante.

Uso Não-Comercial — Você não pode utilizar esta obra com finalidades comerciais.

Compartilhamento pela mesma Licença — Se você alterar, transformar, ou criar outra obra com base nesta, você somente poderá distribuir a obra resultante sob uma licença idêntica a esta.

Para ver uma cópia desta licença, visite http://creativecommons.org/licenses/by-nc-sa/2.5/br/ ou mande uma carta para Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.